Ex. 3.10 (20 pts) — three papers found by search

The three papers below come from Google, Meta and Microsoft, and each answers a question the 1968–2002 literature never faced: DRAM is now the largest single line in server cost, so which pages leave it, and on what evidence? The first two report fleet-wide deployments; the third is a design study on emulated hardware. They also disagree with one another.

1. Software-defined far memory at Google

Andres Lagar-Cavilla, Junwhan Ahn, Suleiman Souhlal, Neha Agarwal, Radoslaw Burny, Shakeel Butt, Jichuan Chang, Ashwin Chaugule, Nan Deng, Junaid Shahid, Greg Thelen, Kamil Adam Yurtsever, Yu Zhao and Parthasarathy Ranganathan, "Software-Defined Far Memory in Warehouse-Scale Computers", *Proc. 24th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*, Providence, RI, April 2019, pp. 317–330. Chosen because it is the first end-to-end report of a cold-page policy running on a whole fleet rather than a testbed.

Summary. A cheap, slow memory tier below DRAM only pays if cold pages can be found without slowing anything down, and a fleet measurement shows why fixed hardware will not do it: at a cold-age threshold of $T = 120\,\text{s}$, 32% of memory is cold on average but the per-machine figure ranges from 1% to 52% within a single cluster, so any provisioned far-memory capacity would be stranded on some machines and exhausted on others. The authors therefore build the tier in software, from `zswap`, which keeps compressed pages in DRAM. Upstream `zswap` fires only on direct reclaim, making its cost unbounded and bursty, so the contribution is a control plane that compresses proactively instead. A kernel daemon `kstaled` sweeps page-table accessed bits every 120 s and keeps an 8-bit age per page inside `struct page`; a second daemon `kreclaimd` compresses pages older than a per-job threshold. The threshold is chosen against a service-level objective expressed in an application-agnostic indicator: the *promotion rate*, or swap-ins per minute, held below 0.2% of the job's working set per minute. A node agent recomputes the smallest threshold meeting that objective each minute and applies the Kth percentile of recent history. The two remaining parameters are tuned offline by replaying the per-job histograms through a model of the whole fleet and searching with a Gaussian Process Bandit. In production since 2016, the system compresses pages 3× at the median with $6.4\,\mu$s decompression, raises cold-memory coverage from 15% to 20%, and saves 4–5% of DRAM total cost of ownership.

Critique. The paper's own motivation sets the bar its evidence cannot quite clear: slowdowns "of even a few percentage points" would "offset all potential TCO savings", yet the strongest quantitative performance result is that Bigtable's IPC difference under A/B test is "within noise" — noise the authors immediately say is expected to be large in a warehouse-scale A/B. Fleet-wide application metrics are reported as showing no statistically meaningful difference, but no resolution is given for that null result, and the resolution is precisely what the 4–5% argument depends on. The accounting is similarly uneven. The quoted CPU overhead, 0.01% and 0.09% of job cycles, covers compression and decompression only; `kstaled`'s continuous scan is reported separately as "less than 11% of a single logical CPU core", in units that never enter the TCO arithmetic. Finally, K and S are never given numeric values anywhere in the paper. The 30% autotuner gain is checkable in itself, being 15% coverage hand-tuned against 20% tuned, but without the two configurations there is no way to tell whether the hand-tuned baseline was a fair opponent, or to reproduce either setting.

2. Transparent memory offloading at Meta

Johannes Weiner, Niket Agarwal, Dan Schatzberg, Leon Yang, Hao Wang, Blaise Sanouillet, Bikash Sharma, Tejun Heo, Mayank Jain, Chunqiang Tang and Dimitrios Skarlatos, "TMO: Transparent Memory Offloading in Datacenters", *Proc. 27th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*, Lausanne, Switzerland, February–March 2022, pp. 609–621. Chosen because it attacks the previous paper directly, with a measurement rather than an argument, and because its mechanisms are upstream in Linux.

Summary. TMO's target is the same cost, but it rejects Google's control signal. The claim is empirical: on the Web application, the host with the *faster* SSD backend shows a *higher* promotion rate and simultaneously serves more requests per second, so swap-in rate is not monotone in application harm, because it ignores both the speed of the backend and the benefit of the memory freed. In its place TMO introduces Pressure Stall Information, a kernel mechanism that measures lost work directly by timing how long processes are stalled on a resource, reported per container as `some` (at least one process stalled) and `full` (all of them). A userspace agent, Senpai, applies the control law $reclaim_mem = current_mem \times r \times \max(0,\, 1 - \mathrm{PSI}_{some}/\mathrm{PSI}_{thresh})$ every six seconds with $r = 0.0005$ and $\mathrm{PSI}_{thresh} = 0.1\%$, so reclaim slows as pressure approaches the target and stops above it. Two kernel changes make the pages available: shadow entries record a page's reuse distance so genuine refaults can be distinguished from first touches, and reclaim is rebalanced to swap anonymous memory as soon as the file cache working set starts refaulting, instead of treating swap as emergency overflow. TMO also offloads the "memory tax" of logging, routing and other sidecars, whose service-level agreements are looser. Across millions of servers it reports 20–32% of total memory saved, with either SSD or `zswap` as the backend.

Critique. That headline is not a measured quantity. It adds the 7–19% of Figure 9 to the 13% of Figure 10, and the paper gives two incompatible accounts of what the first is a fraction of: the caption says "normalized to their resident memory size", per application, while Section 4.1 says "7-19% of the total memory", which is Figure 10's per-server denominator. One of the two is wrong, and until that is settled the sum denotes nothing. The second problem is that TMO's advertised advantage over `g-swap`, that Senpai "does not require offline application profiling", is weaker than stated. The two constants above were "iteratively arrived at" by studying production workloads, and they are global. Section 3.3 asserts that "TMO's effectiveness is not very sensitive to these parameters", and then Section 4.4 shows the opposite: Config B saves more memory than Config A at the cost of "significant RPS regression, violating our design goal". The authors do flag automated tuning as future work, so the defect is the insensitivity claim rather than a missing plan. Last, PSI's IO component treats any wait on block IO as a stall, because the hardware cannot separate oversubscription from a device's expected latency; this reads as pressure exactly where SSDs are cheapest, and the paper's only answer is that it "has worked well for us in production".

3. CXL memory pooling at Microsoft

Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura and Ricardo Bianchini, "Pond: CXL-Based Memory Pooling Systems for Cloud Platforms", *Proc. 28th ACM Int. Conf. on Architectural Support for Programming Languages*

and Operating Systems (ASPLOS '23), Volume 2, Vancouver, BC, March 2023, pp. 574–587. Chosen because it moves the problem out of the kernel's reclaim path entirely and into hardware the other two papers could only wish for.

Summary. In a public cloud the waste is not cold pages but *stranded* ones: traces from 100 Azure clusters show that when all cores of a server are rented, leftover memory cannot be rented, reaching 25% at the 95th percentile. Pooling would fix this, but cloud VMs pin their whole address space so that virtualisation accelerators work, which rules out the page-fault-based schemes of the two papers above. Pond uses Compute Express Link instead, giving cacheable load/store access to an external memory controller shared by 8–16 sockets, a size chosen because wider pools add switches and retimers for diminishing benefit: 8–16 sockets add 70–90 ns over NUMA-local DRAM, rack-scale pooling more than 180 ns. Latency still matters: of 158 workloads under a 182% latency increase, 43% stay within 5% of local-memory performance while 21% lose more than 25%. Two ideas rescue it. Half of all VMs never touch half their rented memory, and that memory can be exposed as a *zNUMA* node, memory but no cores, which the guest's own NUMA policy avoids; measured traffic to it is 0.06–0.38%. Second, two models, a random forest over hardware counters for latency insensitivity and a gradient-boosted quantile regression over VM metadata for untouched memory, predict how much pool memory each VM can take, with a monitor that migrates mispredicted VMs back to local DRAM. The result is 7% less DRAM at a 16-socket pool, or 3.5% of server cost.

Critique. No CXL hardware is involved: the external memory controller is a paper design, its latency is emulated by disabling a second socket's cores, and the savings come from a capacity-only simulator. That is stated plainly, but one consequence is structural: a real 16-socket pool has sixteen hosts contending for one controller's DDR5 channels, whereas the emulation puts a single host on the remote socket, so interference inside the pool is never measured; the paper's answer is that more controllers can be attached. Section 7 does raise gaming, replying that "any individual customer's behavior will have a small impact" across millions of customers. But the untouched-memory model's most important feature is percentiles of untouched memory from *that customer's own* VMs last week, so a customer who touches everything moves their own prediction and nobody else's; the aggregate reply misses the mechanism, and the section's other answer, discounting under-used resources, is a clause the paper never implements or evaluates. The quoted 1–5% degradation likewise holds only inside the configured tail: 2% of VMs exceed the margin and the monitor mitigates at most half of them. What the rest lose is unbounded, with Finding 3 reporting some workloads down 30–35% once a fifth of their memory spills. Pond bounds how many VMs miss the margin, not by how much.